Secure By Design

EDUCATE. EMPOWER. SECURE THE FUTURE.

COURSE 1.4 / COMPANION WORKBOOK

Configure a Root CA & issuing CA

Offline root + subordinate CA / Two-tier PKI

Build a two-tier public key infrastructure with an offline standalone Root CA and an enterprise subordinate/issuing CA. Follow the signing ceremony, publish trust and revocation data, and validate enrollment.

MATERIALS EDITION 1.1 / COURSE 1.4

Updated 8 October 2026 · sbd-two-tier-pki

NAME _____ / DATE _____

START HERE / YOUR LEARNING PATH

Table of contents

Read each lesson alongside the interactive guide and the video. Use the record tables to write down what you actually observe — the workbook is your evidence, not the guide.

Before you start

01 Design the two-tier trust hierarchy

02 Configure the offline Root CA

03 Create the issuing CA request

04 Sign through the offline ceremony

05 Activate issuance and publish trust data

06 Validate enrollment, revocation, and recovery

Course review and sign-off

> **HOW TO USE IT** Each lesson page has the same shape: what you will learn, the build steps, the commands that verify them, and a record table. Fill the table from your own environment — never from the sample values.

Before you start

This course chooses a standalone offline Root CA and an enterprise subordinate/issuing CA for the training build. The root stays disconnected; approved removable media carries requests and public artifacts. The internal publication service must be provisioned separately.

Prerequisites

- ☐ Complete courses 1.1–1.3 (identity, DNS, and the hardened baseline); verify domain health and internal name resolution.
- ☐ Prepare separate Windows Server 2025 hosts: SBD-ROOTCA01 in a disconnected workgroup and SBD-SUBCA01 joined to sbd.test.
- ☐ Approve the CA names, cryptography, validity periods, publication URLs, administration, backups, and removable-media procedure before installation.

What this course builds

- Design the CA hierarchy and its publication dependencies.
- Configure the offline root and sign an issuing CA request through controlled media transfer.
- Publish trust data and prove certificate enrollment and revocation.

Systems in scope

RECORD	YOUR OBSERVATION
SBD-ROOTCA01 — name, address, OS build	
SBD-SUBCA01 — name, address, OS build	

© 2026 Secure By Design Team LLC.

LESSON 01 / ABOUT 25 MIN

Design the two-tier trust hierarchy

> **WHY THIS MATTERS** The root establishes trust. The subordinate, also called the issuing CA in this design, handles approved end-entity certificates. Keeping these roles separate protects the root from daily network exposure.

In this lesson you will

- Explain why the root stays offline while the issuing CA handles enrollment.
- Record the CA identities, cryptography, lifetimes, and CRL periods before installation, starting from this lab's example values.
- Plan AIA and CDP publication locations that clients can reach inside the air gap.

Follow the build

1. Draw the hierarchy: offline standalone root → enterprise issuing CA → approved server/client certificates. There is no network connection to the root.
2. Record final host and CA names, key provider, algorithm, key sizes, hashes, validity periods, renewal owners, and recovery arrangements. Start from this lab's example values in the next step and change any that your own policy requires: they are examples, not universal requirements.
3. Example values for this lab: the root CA uses an RSA 4096 key with SHA-256 and a 20-year certificate, and publishes a base CRL every 26 weeks with an overlap period and no delta CRL, refreshed at each offline ceremony. The issuing CA uses RSA 4096 (3072 at minimum) with SHA-256 and a 10-year certificate, a weekly base CRL, and a daily delta CRL. End-entity certificates last one to two years. NIST SP 800-57 Part 1 rates RSA 2048 at 112 bits of security, which it accepts for new signatures only through 2030, so a CA that will still be signing after 2030 uses at least 3072 bits (128 bits of security); Windows Server 2025 requires at least 2,048-bit RSA for TLS server authentication.
4. Know where each value is set. You choose the key, the hash, and the root's 20-year validity in the AD CS Configuration wizard. Each CA's CAPolicy.inf sets its CRL periods (CRLPeriod, CRLPeriodUnits, CRLDeltaPeriod, CRLDeltaPeriodUnits), which certutil -setreg can change later. The issuing CA's 10-year lifetime is set on the root (CA\ValidityPeriod and CA\ValidityPeriodUnits) before the signing ceremony, because a subordinate CA's lifetime comes from the CA that signs it.
5. Plan a stable internal publication name such as pki.sbd.test. The Authority Information Access (AIA) location supplies issuer certificates; the CRL Distribution Point (CDP) supplies certificate revocation lists.
6. Choose the publication host, paths, access rules, and refresh schedule before issuing certificates. Clients inside this air-gapped network must reach those locations without relying on the public internet.

Check your result

```
# Run on the future issuing CA, not the offline root.
Get-CimInstance Win32_ComputerSystem | Select-Object Name, Domain, PartOfDomain
Resolve-DnsName SBD-DC01.sbd.test
Resolve-DnsName SBD-DC02.sbd.test
```

> **WHAT SUCCESS LOOKS LIKE** The written plan separates trust, issuance, publication, and administration. It identifies who renews CRLs before they expire, and every key size, hash, lifetime, and CRL period has a recorded value and the setting that applies it.

Your record

Evidence to keep: Keep the approved hierarchy, the cryptography and lifetime table with the setting that applies each value, and a worksheet mapping every certificate/CRL URL to its actual publisher and file.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Which role issues routine server certificates in this design?

REMEMBER Trust, issuance, publication, and administration are separated in writing. Cryptography, names, lifetimes, and CRL periods are approved before installation, starting from this lab's example values. CRL renewal has a named owner and a date.

Microsoft · PKI design considerations

Microsoft · CAPolicy.inf syntax

NIST · SP 800-57 Part 1 Rev. 5, Recommendation for key management

LESSON 02 / ABOUT 30 MIN

Configure the offline Root CA

> **WHY THIS MATTERS** Treat root installation as a controlled event. Its private key is the highest-value secret in the hierarchy, and its configuration shapes the subordinate certificate it will sign.

In this lesson you will

- Verify the disconnected workgroup host before installing the role.
- Install a standalone root CA with the approved CAPolicy.inf and settings.
- Configure publication URLs and the CRL schedule before signing anything.

Follow the build

1. Verify SBD-ROOTCA01 is in a workgroup with no active network path. Prepare approved installation media, protected administration, correct time, and backup storage.
2. Place the reviewed CAPolicy.inf in %SystemRoot% before configuration. Tailor it to the root role: in this lab it sets CRLPeriod=Weeks and CRLPeriodUnits=26, and CRLDeltaPeriodUnits=0 so the root publishes no delta CRL. A self-signed root's own CDP/AIA choices differ from the extensions the root places into issued subordinate certificates.
3. Install the Certification Authority role with management tools. In the AD CS Configuration wizard choose Standalone CA, Root CA, and a new private key, then apply the approved provider, cryptography (RSA 4096 with SHA-256 in this lab), CA name, validity (20 years), database, and log settings.
4. Before signing anything, configure the root's issued-certificate AIA/CDP settings, the subordinate validity policy (CA\ValidityPeriod Years and CA\ValidityPeriodUnits 10 in this lab), and the base-CRL schedule (26 weeks with an overlap period, and no delta CRL). Hold requests for explicit approval. Produce and inspect a current root CRL, then capture a protected CA backup.

Check your result

```
# Inspect locally on SBD-ROOTCA01 during the approved root session.
Get-CimInstance Win32_ComputerSystem | Select-Object Name, Domain, PartOfDomain
Get-Service CertSvc
certutil -getreg CA\CRLPublicationURLs
certutil -getreg CA\CACertPublicationURLs
certutil -getreg CA\ValidityPeriodUnits
certutil -getreg CA\CRLPeriodUnits
```

> **WHAT SUCCESS LOOKS LIKE** The host remains disconnected, the CA is standalone/root, requests require approval, and the planned subordinate URLs and 10-year subordinate validity are configured before signing.

Your record

Evidence to keep: Record the root certificate identity, policy file, publication settings, CRL dates, backup location, and operator sign-off. Protect backups containing private keys.

RECORD	YOUR OBSERVATION
Host(s) and time collected	

RECORD	YOUR OBSERVATION
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

When must the root's issued-certificate publication URLs be configured?

> **REMEMBER** The root host stays disconnected, with controlled media, administration, and time. CAPolicy.inf is in place before the role is configured. Publication URLs, validity policy, the CRL schedule, and a protected backup exist before any signing.

Microsoft · CAPolicy.inf syntax

Microsoft · Install the offline root

Microsoft · Perform the offline root signing ceremony

LESSON 03 / ABOUT 25 MIN

Create the issuing CA request

> **WHY THIS MATTERS** Prepare the online issuing server, but keep issuance paused until its CA certificate has been signed and installed. The request carries the public key; the private key stays protected on the issuing system or its approved key provider.

In this lesson you will

- Prepare the domain-joined issuing host with its final identity.
- Generate the subordinate CA request without exposing the private key.
- Verify and hash the request before it travels to the root.

Follow the build

1. Confirm SBD-SUBCA01 has its final hostname, domain membership, static address, internal DNS, and correct time. Use a dedicated host and the privileges approved for enterprise CA configuration.
2. Prepare its role-specific CAPolicy.inf (in this lab: CRLPeriod=Weeks, CRLPeriodUnits=1, CRLDeltaPeriod=Days, CRLDeltaPeriodUnits=1, and LoadDefaultTemplates=0) and protected request directory. Decide which templates may later be published instead of enabling a broad default enrollment set.
3. Install the CA role and management tools. In the AD CS Configuration wizard choose Enterprise CA, Subordinate CA, and the approved new-key/provider settings (RSA 4096 with SHA-256 in this lab). Enter the planned CA identity and storage locations.
4. Choose Save a certificate request to file. Inspect the resulting request's subject, public key, and CA constraints; record its SHA-256 hash before handing it to the media custodian.

Check your result

```
# Use the actual request path chosen in the wizard.
certutil -dump C:\CAConfig\IssuingCA.req
Get-FileHash -Algorithm SHA256 C:\CAConfig\IssuingCA.req
```

> **WHAT SUCCESS LOOKS LIKE** The request matches the approved issuing identity and key profile. The CA is waiting for its signed certificate; do not troubleshoot that expected state by installing another CA.

Your record

Evidence to keep: Retain the request, its reviewed fields and hash, and the request-transfer record. Do not place private keys in the transfer package.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	

© 2026 Secure By Design Team LLC.

RECORD	YOUR OBSERVATION
Evidence file names	
Open items or findings	

Think it through

What moves to the offline root for signing?

> **REMEMBER** The issuing host has its final name, addressing, DNS, and time before the role install. The request file carries the public key only. The request's fields and hash are reviewed and recorded before transfer.

Microsoft · CAPolicy.inf syntax

Microsoft · Deploy the issuing CA role

LESSON 04 / ABOUT 30 MIN

Sign through the offline ceremony

WHY THIS MATTERS A signing ceremony connects an approved request to an approved CA identity. A matching filename alone is not proof that the root should trust the request.

In this lesson you will

- Verify request integrity at the root before submission.
- Issue the subordinate certificate under the approved validity policy.
- Export and verify the public artifacts, then return the root to a secured state.

Follow the build

1. Use approved removable media and custody procedures to bring the request to the root. Recalculate its hash and compare it with the independently recorded value; inspect the request before submission.
2. Verify the root's issued-certificate validity policy and publication extensions. In Certification Authority, submit the request, review its pending entry, and issue it only after the required approval.
3. Export the subordinate certificate, root certificate, and current root CRL. Check identities, constraints, validity, signatures, URLs, and revocation data against the approved hierarchy. Record file hashes and the request ID.
4. Transfer only the approved public artifacts back to the connected environment. Match published filenames to the URLs embedded in certificates, retain the ceremony record and protected backup, then shut down and secure the root.

Check your result

```
# Inspect public artifacts; use your actual approved paths.
certutil -dump C:\CAConfig\IssuingCA.cer
certutil -dump C:\CAConfig\RootCA.crl
Get-FileHash -Algorithm SHA256 C:\CAConfig\IssuingCA.cer, C:\CAConfig\RootCA.cer, C:\
CAConfig\RootCA.crl
```

WHAT SUCCESS LOOKS LIKE Artifact identities, signatures, validity, and custody checks pass. The root remains offline; a field dump alone is not end-to-end trust validation.

Your record

Evidence to keep: Keep approval, request ID, artifact hashes, verified certificate/CRL properties, media custody, and root shutdown evidence together.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	

RECORD	YOUR OBSERVATION
Evidence file names	
Open items or findings	

Think it through

Which transfer preserves the root boundary?

REMEMBER Integrity is verified before submission, and issuance follows explicit approval. Artifact identities, validity, and URLs are checked against the plan. The ceremony record, hashes, and root shutdown are all evidence.

Microsoft · Perform the offline root signing ceremony

LESSON 05 / ABOUT 30 MIN

Activate issuance and publish trust data

WHY THIS MATTERS A running CA is only one part of a working PKI. Clients also need the approved trust anchor, the intermediate certificate, and current revocation information at the advertised locations.

In this lesson you will

- Install the signed certificate and start Certificate Services.
- Publish certificates and CRLs at the exact advertised URLs.
- Publish only the certificate templates the lab requires.

Follow the build

1. Verify the returned artifacts and import the approved root certificate into the issuing server's trusted root store. Complete CA certificate installation through Certification Authority and start Certificate Services.
2. Configure the issuing CA's AIA/CDP settings before issuing test certificates. Publish current root and issuing certificates and CRLs to the planned internal locations, preserving the actual filenames referenced by their extensions.
3. Distribute the approved root trust through the managed domain policy and arrange the required intermediate/directory publication with delegated administrators. Public CA certificates and CRLs may be readable to clients; private keys and backups must never enter that web directory.
4. Publish only the certificate templates required for the lab. Review purpose, subject construction, key settings, validity, and enrollment permissions; grant a small test group the minimum enrollment rights first.

Check your result

```
# Inspect the issuing CA after installation and publication.
Get-Service CertSvc
certutil -getreg CA\CRLPublicationURLs
certutil -getreg CA\CACertPublicationURLs
certutil -store Root
certutil -store CA
```

WHAT SUCCESS LOOKS LIKE Issuance is configured, the approved trust chain reaches the client, and the exact published URLs return the current files from the internal network.

Your record

Evidence to keep: Keep publication mappings, permissions, template settings, trust-distribution policy, and retrieval results.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	

© 2026 Secure By Design Team LLC.

RECORD	YOUR OBSERVATION
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Which files belong in the client-readable PKI publication directory?

> **REMEMBER** The trust chain and current CRLs are reachable at the advertised internal URLs. Root trust distributes through managed domain policy. Only required templates are published, with minimal enrollment rights.

Microsoft · Publish trust data and validate clients

Microsoft · Certutil reference

LESSON 06 / ABOUT 25 MIN

Validate enrollment, revocation, and recovery

WHY THIS MATTERS Finish from a clean domain client. Verify the experience of a relying system, including what happens when a certificate is no longer trusted.

In this lesson you will

- Enroll and inspect a disposable test certificate.
- Verify chain building and revocation retrieval from a clean client.
- Prove revocation works, and schedule the renewals that keep the PKI alive.

Follow the build

1. Enroll a disposable test certificate using the approved template. Export only its public certificate as test.cer and inspect the subject, purpose, issuer, and validity.
2. On a clean validation client, verify the chain and retrieval using the command below. Inspect each AIA/CDP URL and current CRL dates; cached success is insufficient evidence of publication reachability.
3. Revoke only the disposable test certificate, publish an updated issuing CRL, and verify that a fresh client evaluation detects revocation. Diagnose retrieval or stale-publication problems rather than turning off revocation checks.
4. Record owners and reminders for root/issuing CRL publication, certificate renewal, backup, and an isolated recovery rehearsal. Schedule offline root ceremonies early enough to transfer and publish replacement CRLs before expiry.

Check your result

```
# On the validation client; test.cer is the disposable public certificate.
certutil -dump test.cer
certutil -urlfetch -verify test.cer
```

WHAT SUCCESS LOOKS LIKE A valid test certificate chains to the approved root and retrieves current revocation data. After revocation and publication, the test certificate is rejected for the expected reason.

Your record

Evidence to keep: Finish the PKI checklist with enrollment, retrieval and revocation results, renewal dates, backup verification, and recovery ownership.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	

RECORD	YOUR OBSERVATION
Evidence file names	
Open items or findings	

Think it through

A valid certificate fails because its CRL cannot be retrieved. What is the next step?

> **REMEMBER** A clean client builds the chain and retrieves live revocation data. A revoked test certificate is rejected for the expected reason. Renewal, backup, and recovery each have an owner and a date.

Microsoft · Publish trust data and validate clients

Microsoft · Certutil reference

Course review and sign-off

Before moving on, confirm each lesson left evidence behind — and that what remains open is written down, not remembered.

Lesson completion

LESSON	EVIDENCE FILED	DATE
01 Design the two-tier trust hierarchy	☐ yes	
02 Configure the offline Root CA	☐ yes	
03 Create the issuing CA request	☐ yes	
04 Sign through the offline ceremony	☐ yes	
05 Activate issuance and publish trust data	☐ yes	
06 Validate enrollment, revocation, and recovery	☐ yes	

Open items carried forward

Next in the learning path

That completes Configure a Root CA & issuing CA. Course 1.5 connects and segments the network, and course 1.6 then finishes the PKI services: the pki.sbd.test publication site and template administration.

> **KEEP IT HONEST** Lesson completion records learning activity; it is not a system authorization decision. Your observations, not the sample values, establish what happened in your environment.

© 2026 Secure By Design Team LLC.